



PDDL & High-Level Actions

Tutorial 1 • Week 3 • Real-World Planning and Acting

CS4246 / CS5446 | AY2026/27 Semester 1

Representation → search → relaxation → hierarchy

jing.yu@nus.edu.sg



00 Logic Survival Kit

Read the symbols, distinguish objects from predicates, and interpret states, goals, and grounded actions.

QUESTION → DISCUSSION → REVEAL

■ What does each formula say?

TRANSLATE INTO ENGLISH

1

$\text{Clear}(A) \wedge \text{On}(A, B)$

?

2

$\neg \text{Clear}(B)$

?

3

$\text{Clear}(A) \vee \text{Clear}(B)$

?

4

$A \neq B$

?

Say the connective aloud before interpreting the whole formula.

■ A few symbols carry most of the tutorial

AIMA 4e §8.2; §11.1

[ANSWER / DISCUSS](#)

$\wedge$

and

both parts must hold

$\neg$

not

the following atom is false

$\vee$

or

at least one part holds

$\rightarrow$

implies

if the left holds, the right must hold

$= / \neq$

same / different

compare object identities

Basic STRIPS mostly uses conjunctions of positive or negative literals.

■ Classify the logic vocabulary

CLASSIFY EACH ITEM

A

x

On/2

On(x, y)

On(A, B)

$\neg$ Clear(B)

Which items may contain variables? Which ones can be true or false in a state?

■ Objects fill predicate argument slots

[ANSWER / DISCUSS](#)

A	OBJECT CONSTANT	names one particular object
x	VARIABLE	a placeholder for an object
On/2	PREDICATE SYMBOL	a two-argument relation
On(x,y)	ATOMIC FORMULA	an atom that still has variables
On(A,B)	GROUND ATOM	a proposition that can be true or false
¬Clear(B)	NEGATIVE LITERAL	a negated ground atom

A fluent is a ground atom whose truth value may change as actions execute.

■ Quantifiers define who a formula covers

$\forall x$

for every x

$\forall x [\text{Block}(x) \rightarrow x \neq \text{Table}]$

$\exists x$

there exists an x

$\exists x [\text{Block}(x) \wedge \text{Clear}(x)]$

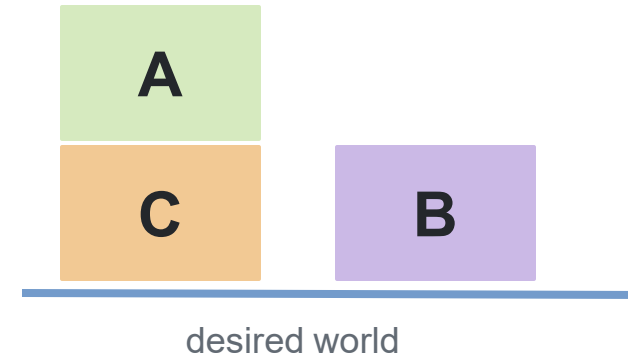
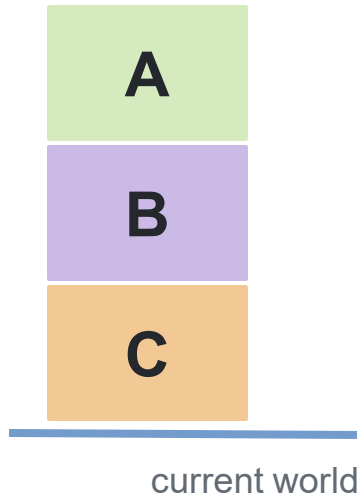
**Scope matters: x is bound only inside the brackets.
PDDL action parameters are placeholders that are later grounded - not claims that every object satisfies the action.**

Quick check: which formula says that at least one block is clear?

■ Before PDDL: define the planning problem

OPENING QUESTION

A robot hears: “put the middle block on the table.”



What must the agent know before search can even begin?

Name 3 pieces of information.

■ Planning starts by making choices explicit

ANSWER / DISCUSS

1

STATE

What is true now?

2

ACTIONS

What can change?

3

GOAL

What counts as
success?

4

SEARCH

Which sequence
works?

Today: model precisely, reason aloud, and use abstraction deliberately.

Reading route: AIMA 4e §11.1 → §11.3 → §11.4

■ Quick check: when is planning “classical”?

VOTE BEFORE REVEAL

Which assumptions belong to classical planning?

- | | |
|--------------------------------------|---------------------------------------|
| A Fully observable | B Partially Observable |
| C Deterministic | D Stochastic |
| E Continuous States/Actions | F Discrete States/Actions |
| G Static world while planning | F Dynamic world while planning |

■ Classical planning simplifies on purpose

AIMA 4e §11.1

[ANSWER / DISCUSS](#)

OBSERVATION

Fully observable

The current state is known.

DYNAMICS

Deterministic

An action has a specified result.

TIME

Static

The world does not change while we plan.

REPRESENTATION

Discrete

States and actions are symbolic choices.

These are modeling assumptions – for now...

■ PDDL: reusable rules, concrete tasks

DOMAIN.PDDL

- requirements and types
- predicate declarations
- action schemas

PROBLEM.PDDL

- objects
- initial state
- goal description

same domain, many problems

■ Schema, action, state, or goal?

MATCH THE TERMS

- 1 `Move(x, from, to)`
- 2 `Move(A, B, Table)`
- 3 `{On(A,B), On(B,C), Clear(A)}`
- 4 `On(B,Table)  $\wedge$  On(A,C)`

- action schema
- grounded action
- state
- goal description

Which item may contain variables?

■ Lifted schemas, grounded actions

AIMA 4e §11.1

[ANSWER / DISCUSS](#)

- 1 Move(x, from, to)
- 2 Move(A, B, Table)
- 3 {On(A,B), On(B,C), Clear(A)}
- 4 On(B,Table) $\wedge$ On(A,C)

ACTION SCHEMA

variables define a family

GROUNDING ACTION

objects instantiate the schema

STATE

ground fluents that are true

GOAL

a partial description of success

AIMA reminder: database semantics uses the closed-world and unique-names assumptions.

■ Grounding replaces variables with objects

ACTION SCHEMA

`Move(x, fr, to)`

SUBSTITUTION

$\theta = \{x \mapsto A, fr \mapsto Table, to \mapsto C\}$

GROUNDING ACTION

`Move(A, Table, C)`

Apply the same substitution to the preconditions

`Clear(A)  $\wedge$  Block(A)  $\wedge$  On(A, Table)  $\wedge$  Clear(C)  $\wedge$  Block(C)`

The grounded action is applicable only if every grounded precondition is true in the current state.

■ One action: what survives and changes?

WORK IT OUT

```
s = {At(P1,SFO), Plane(P1), Airport(SFO), Airport(SIN)}  
  
Fly(P1,SFO,SIN)  
PRE: At(P1,SFO) ∧ Plane(P1) ∧ Airport(SFO) ∧ Airport(SIN)  
EFF: ¬At(P1,SFO) ∧ At(P1,SIN)
```

Write the next state s' .

Which facts are unchanged - and why?

■ PDDL changes only the listed fluents

AIMA 4e Eq. 11.1

[ANSWER / DISCUSS](#)

$$\text{RESULT}(s,a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$$

DELETE
At(P1,SFO)

ADD
At(P1,SIN)

PERSIST
Plane / airports

$s' = \{\text{At}(P1,\text{SIN}), \text{Plane}(P1), \text{Airport}(\text{SFO}), \text{Airport}(\text{SIN})\}$

This compactly solves the frame problem.

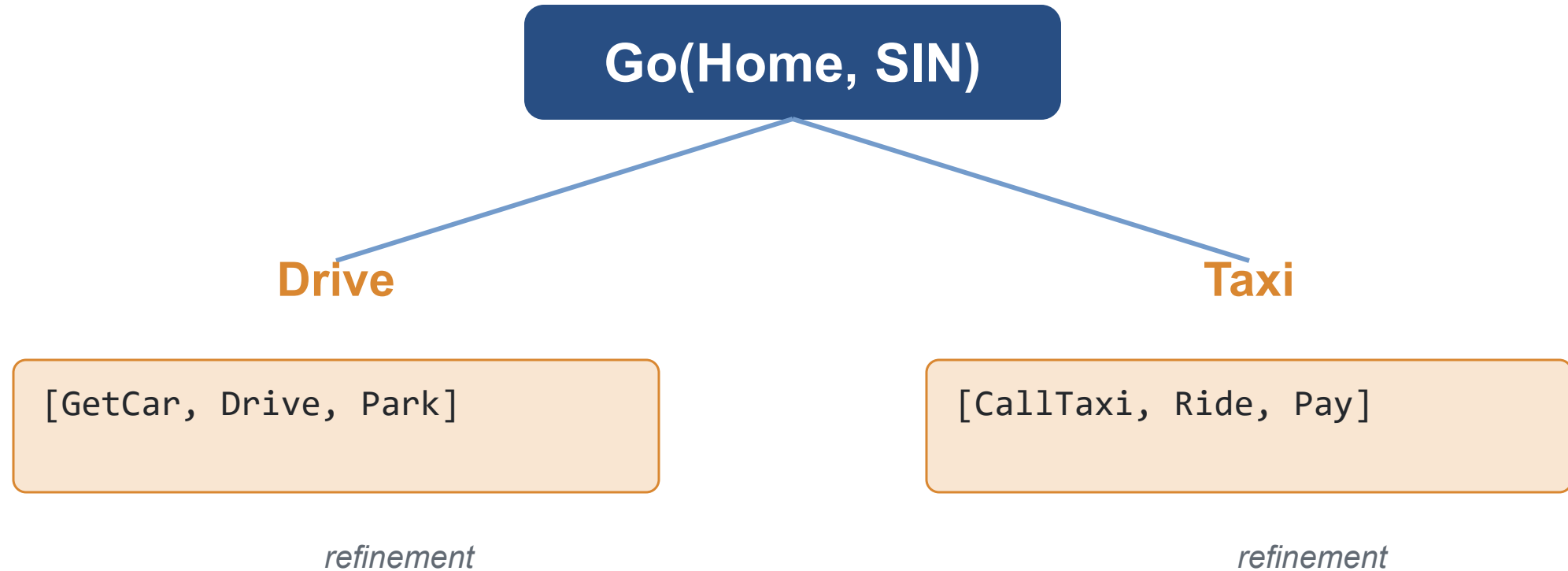
■ Forward search turns planning into tree search

AIMA 4e §11.2

CHECK THE MAPPING

NODE	=?	state
EDGE	=?	applicable grounded action
GOAL TEST	=?	state satisfies goal
PATH	=?	plan

■ HLAs encode reusable methods



An implementation contains primitive actions only.

■ Quick check: what do possible effects mean?

AIMA 4e §11.4.2

SAY IT IN WORDS

$\sim +A$



?

$\sim -A$



?

$\sim \pm A$



?

■ Possible effects summarize implementations

AIMA 4e §11.4.2

[ANSWER / DISCUSS](#)

$\sim +A$

possibly add A

leave A unchanged OR make A true

$\sim -A$

possibly delete A

leave A unchanged OR make A false

$\sim \pm A$

possibly add or delete A

an implementation may control A either way

The agent chooses the implementation: this is angelic semantics, not stochastic dynamics.



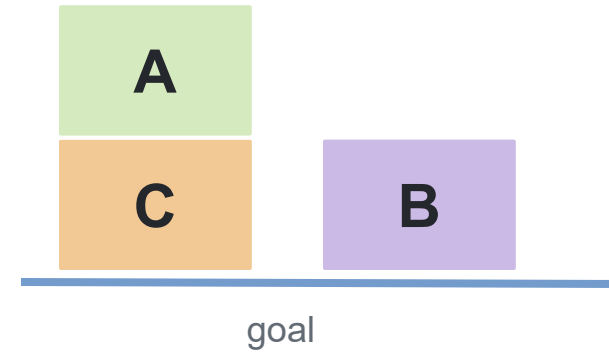
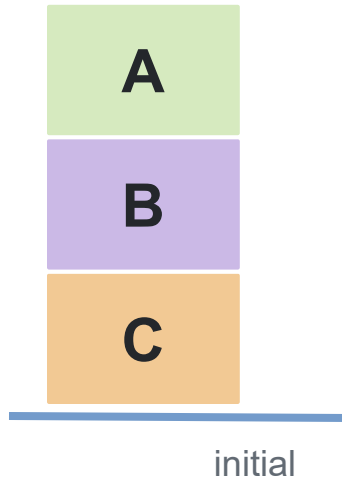
01 A Blocks World

Model Middle-pop, expand a forward search tree, and read a plan as state change.

QUESTION → DISCUSSION → REVEAL

■ Problem 1A: what facts describe Middle-pop?

DO NOT WRITE PDDL YET



1. What is true initially?

2. What must be true at the goal?

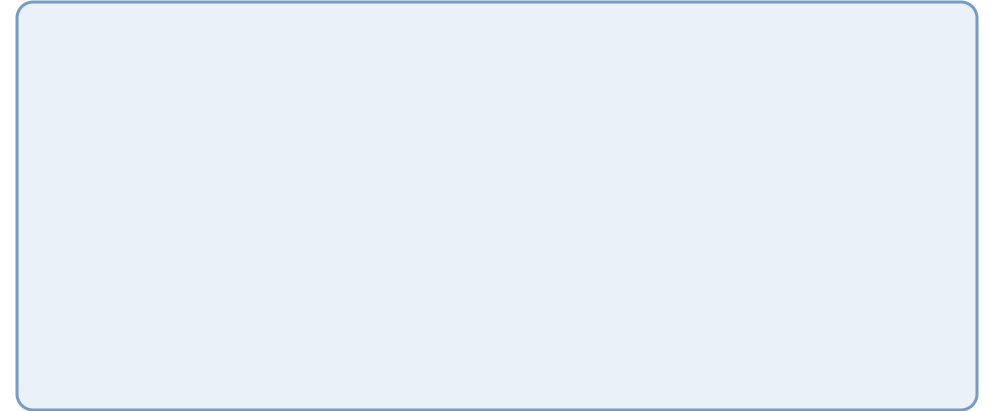
3. Which modeling assumption makes the table special?

■ Middle-pop: state, goal, and table convention

[ANSWER / DISCUSS](#)

INITIAL STATE

GOAL



Predicates: `Block(x)`, `On(x,y)`, `Clear(x)`

Convention: `Clear(Table)` is always true - the table has unlimited clear space.

■ Problem 1A: action schema?

DESIGN CHOICE

Could Move(x, from, to) handle every destination - including Table?

OPTION A

One general Move
with conditional table logic

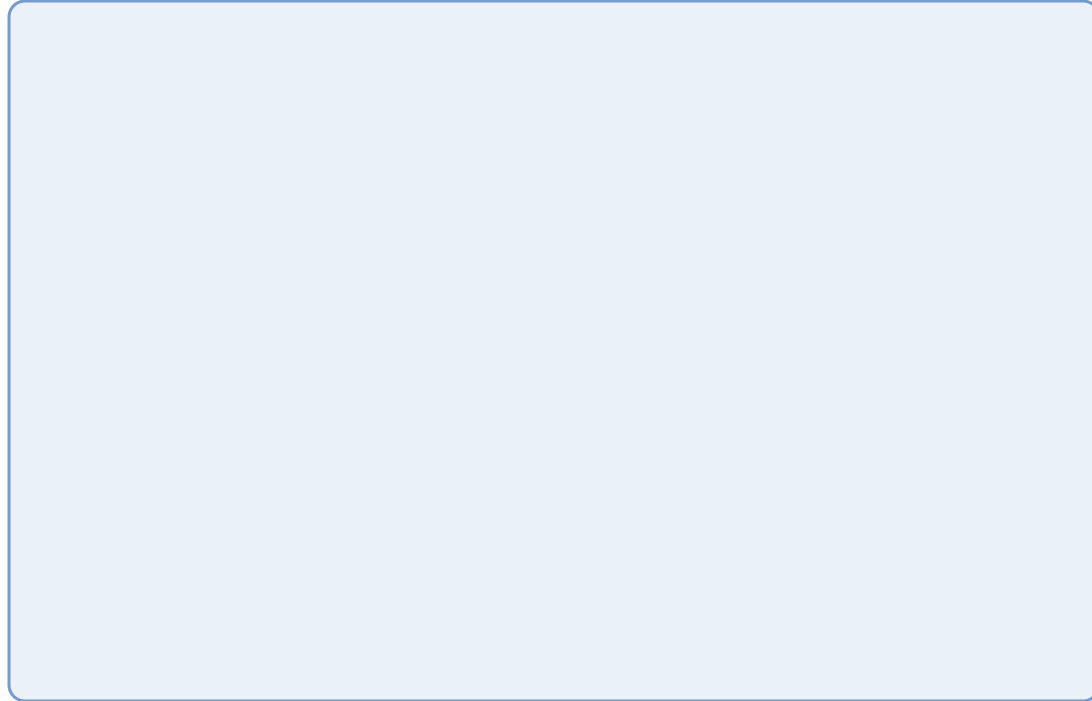
OPTION B

Move onto a block
+ MoveToTable

■ Two schemas make the table exception explicit

[ANSWER / DISCUSS](#)

Move(x, fr, to)



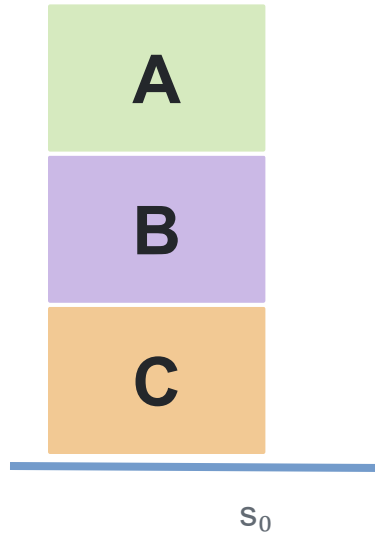
MoveToTable(x, fr)



Type constraints prevent moving Table and prevent stacking onto Table through Move.

■ Problem 1B: what actions apply in s_0 ?

APPLICABILITY CHECK



Candidate grounded actions

- MoveToTable(A,B)
- MoveToTable(B,C)
- Move(A,B,C)
- Move(A,B,Table)
- Move(C,Table,A)

Which precondition eliminates each illegal action?

■ Only the top block can move first

[ANSWER / DISCUSS](#)

Applicable:

MoveToTable(A,B)

MoveToTable(B,C)

✗

fails Clear(B)

Move(A,B,C)

✗

fails Clear(C); destination occupied

Move(A,B,Table)

✗

wrong schema: Move requires Block(to)

Move(C,Table,A)

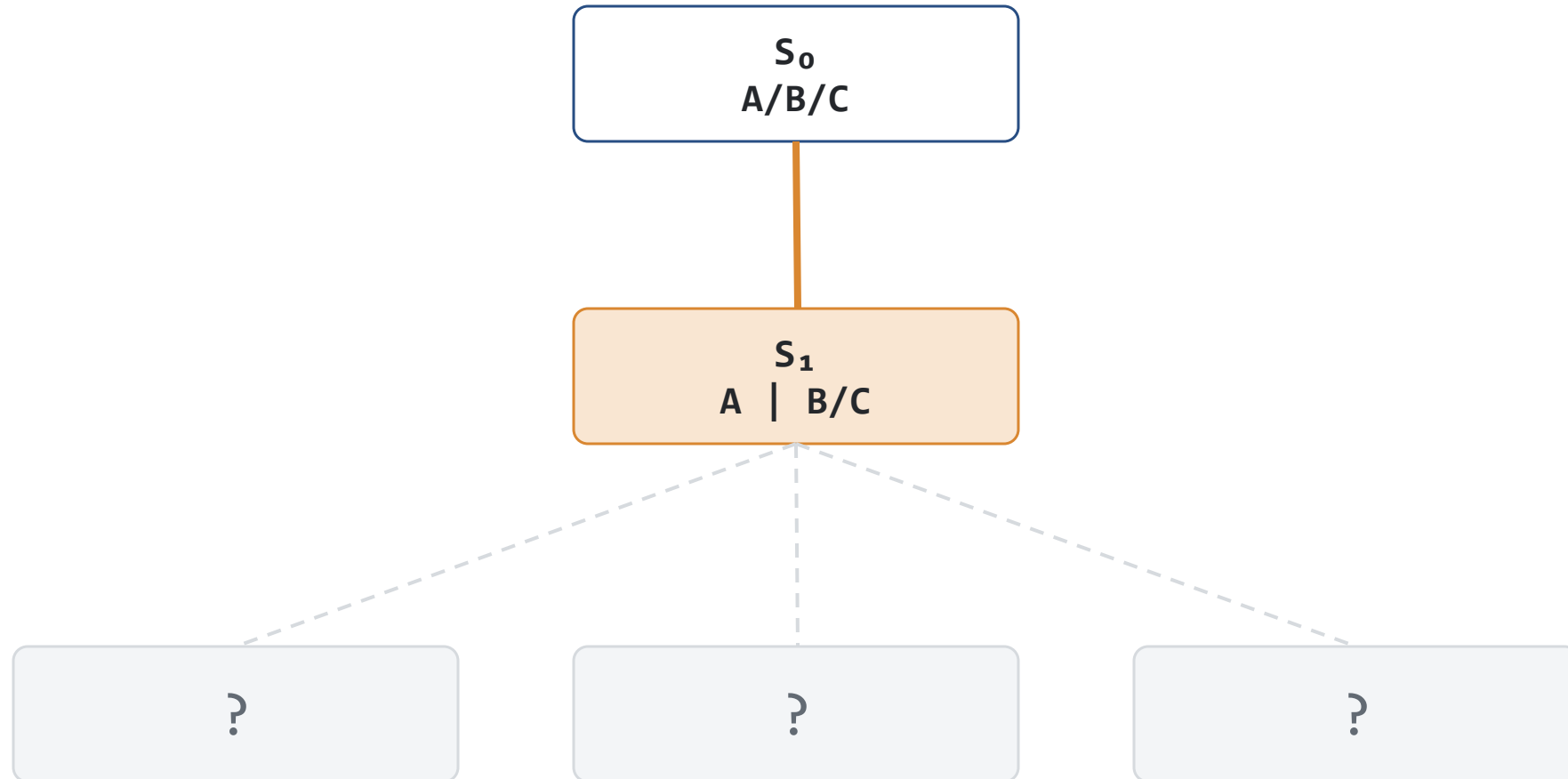
✗

fails Clear(C)

Search branching comes from applicable grounded actions - not from every schema instance.

■ Problem 1B: draw two search-tree levels

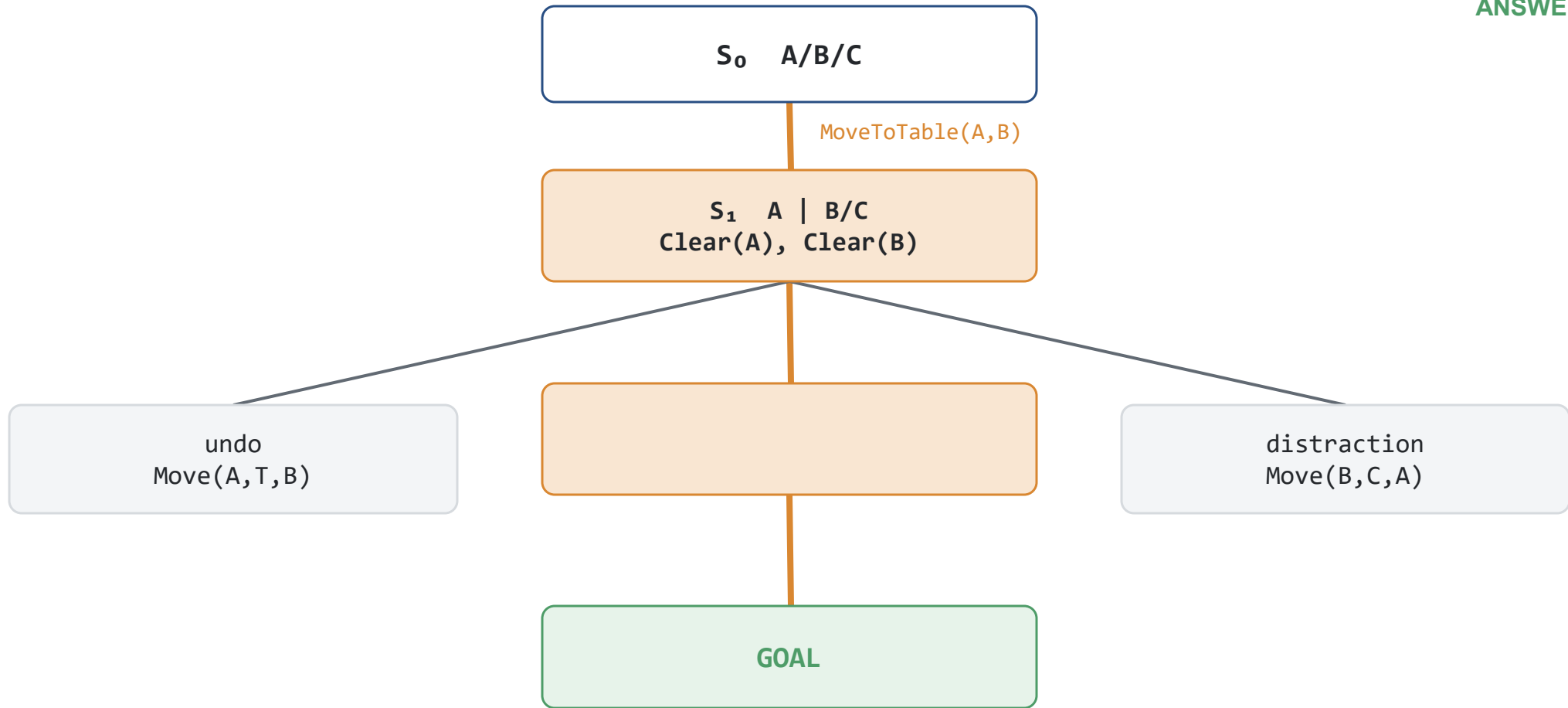
BOARD WORK



Label every edge with an action and every node with the resulting fluents.

Forward tree: progress, distractions, and undo

ANSWER / DISCUSS

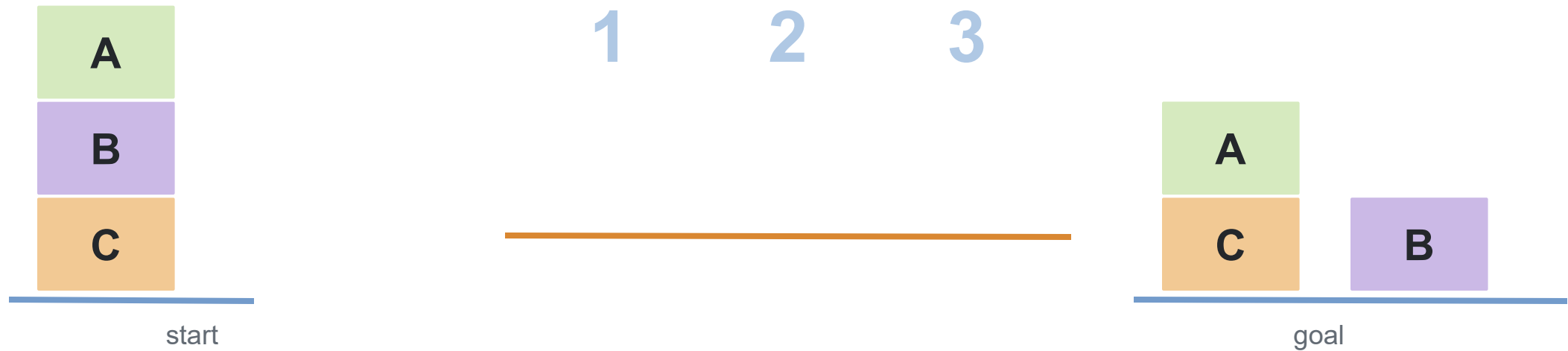


The goal branch appears only after B moves off C.

Selective tree: declare omitted symmetric or repeated branches.

■ Problem 1C: find a three-step plan

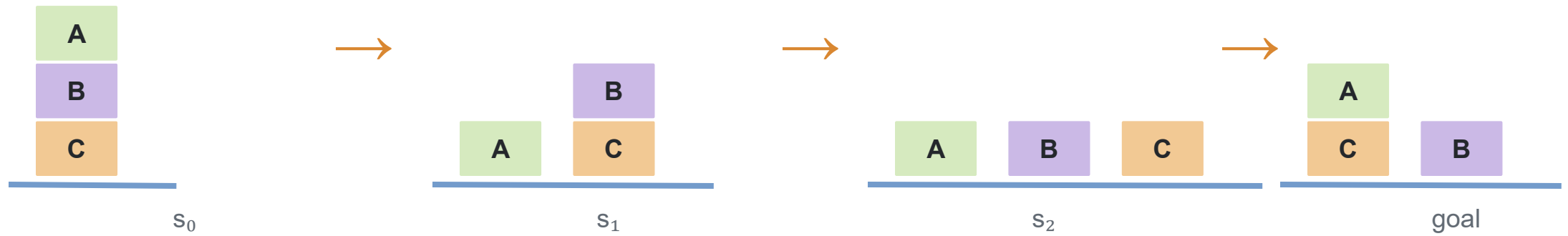
PLAN BEFORE REVEAL



Constraint: justify each action's preconditions in the state where it executes.

■ Clear B, remove it, then restore A

ANSWER / DISCUSS



$\pi =$

Each intermediate state satisfies the next action's preconditions.



02 Delete Relaxation

Understand why ignoring negative effects can guide search - and exactly when the argument breaks.

QUESTION → DISCUSSION → REVEAL

■ Problem 2: why is ignore-delete a relaxation?

AIMA 4e §11.3

ARGUE, DON'T MEMORIZE

Assume all goals and action preconditions are positive literals.

Original: $s' = (s - \text{DEL}(a)) \cup \text{ADD}(a)$
Relaxed: $s' = s \cup \text{ADD}(a)$

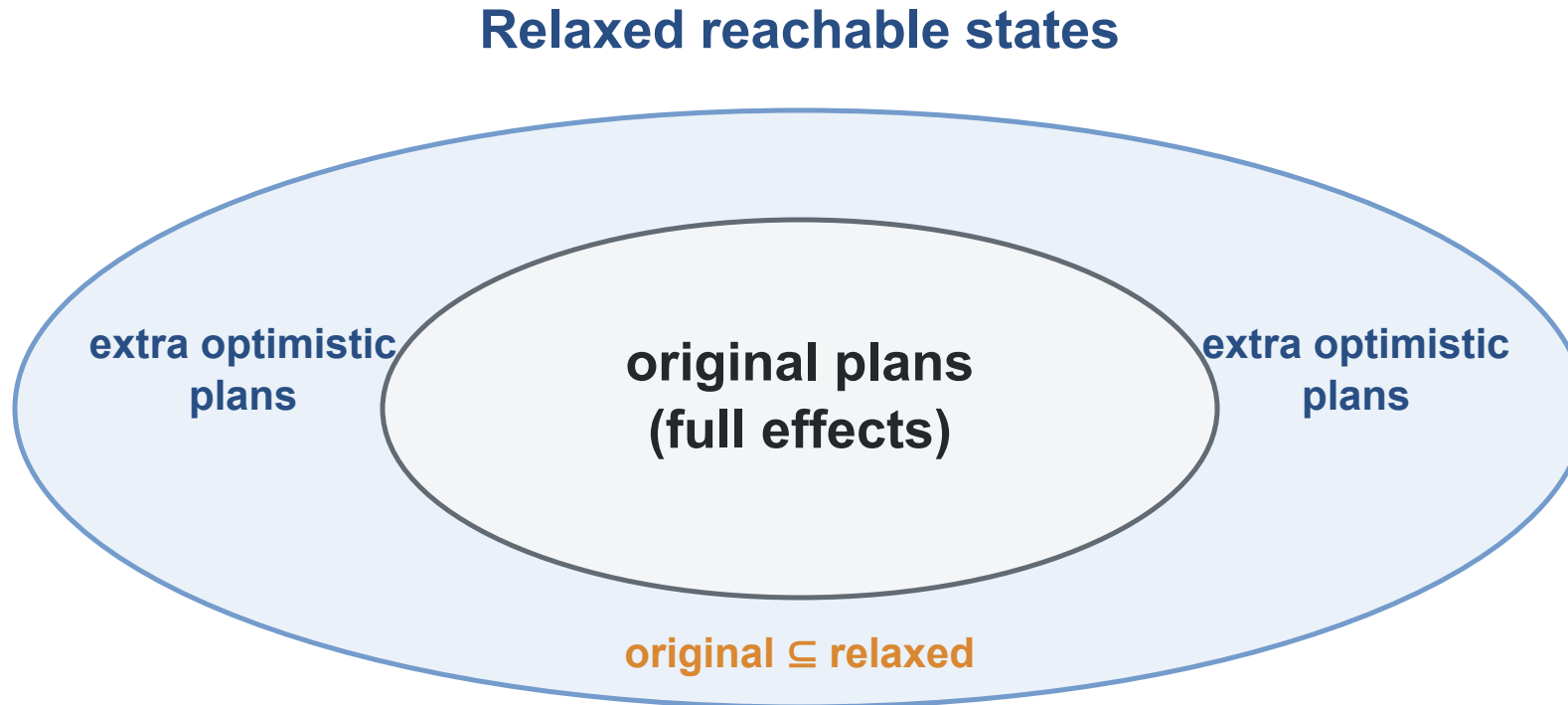
Complete this sentence:

“Deleting a positive fact can never _____ a positive precondition or goal.”

■ With positive conditions, facts can only help

AIMA 4e §11.3

[ANSWER / DISCUSS](#)



Removing DEL cannot remove an original solution; it may introduce easier ones.

Therefore relaxed optimal cost $\leq$ original optimal cost.

■ Does ignore-delete survive a negative goal?

COUNTEREXAMPLE

```
Init:  {At(RoomA)}  
Goal:  At(RoomB)  $\wedge$   $\neg$ At(RoomA)  
  
Move(RoomA, RoomB)  
PRE:   At(RoomA)  
ADD:   At(RoomB)  
DEL:   At(RoomA)
```

**If we delete the delete
list...**

**can the relaxed
problem still reach
the goal?**

YES / NO - justify

■ Negative conditions can require delete effects

AIMA 4e §11.3

[ANSWER](#) / [DISCUSS](#)

ORIGINAL

After Move:
 $\text{At}(\text{RoomB}) \wedge \neg \text{At}(\text{RoomA})$

✓ goal reached

IGNORE DELETE

After Move:
 $\text{At}(\text{RoomA}) \wedge \text{At}(\text{RoomB})$

■ Relaxed plans become search heuristics

AIMA 4e §11.3

real state  relax relaxed problem  extract heuristic $h(s)$

Fast Forward (FF)

- ignore delete effects
- greedily extract a relaxed plan
- use its action count to guide search

Optimistic structure can be computationally useful even when the relaxed plan is not executable.



03 Hierarchical Task Planning

Summarize implementations into HLA schemas, then test concrete refinements from the initial state.

QUESTION → DISCUSSION → REVEAL

■ Problem 3: organize primitive actions

make A / choose B

P1: $\neg A \rightarrow A$
P2: $A \rightarrow B$
P3: $A \rightarrow \neg B$

control C when $\neg B$

P4: $\neg B \rightarrow A \wedge C$
P5: $\neg B \rightarrow C$
P6: $\neg B \rightarrow A \wedge \neg C$
P7: $\neg B \rightarrow \neg C$

reach D then E

P8: $\neg B \wedge \neg C \rightarrow D$
P9: $D \rightarrow E$
P10: $\neg B \wedge D \rightarrow E$

**H1 summarizes alternatives for B; H2 summarizes alternatives for A and C;
H3 composes D then E.**

■ Problem 3A(i): summarize H1's implementations

DERIVE THE NET EFFECT

H1 precondition: $\neg A$

Implementation 1
[P1, P2]
 $\neg A \rightarrow A \rightarrow B$

Implementation 2
[P1, P3]
 $\neg A \rightarrow A \rightarrow \neg B$

What is guaranteed? What may vary?

H1 effect = _____

■ Problem 3A(ii-iii): derive H2 and H3

FIND COMMON PRECONDITIONS + NET
EFFECTS

H2

[P4] [P5] [P6] [P7]

All require: _____

Guaranteed / possible: _____

P4: $\neg B \rightarrow A \wedge C$

P5: $\neg B \rightarrow C$

P6: $\neg B \rightarrow A \wedge \neg C$

P7: $\neg B \rightarrow \neg C$

■ Problem 3A(ii-iii): derive H2 and H3

FIND COMMON PRECONDITIONS + NET EFFECTS

reach D then E

P8: $\neg B \wedge \neg C \rightarrow D$
P9: $D \rightarrow E$
P10: $\neg B \wedge D \rightarrow E$

H3

[P8,P9] [P8,P10]

All require: _____
Guaranteed: $D \wedge E$

Careful: a sequence's precondition includes what must hold before its first action.

■ Problem 3B: which refinements reach E?

CHOOSE ALL THAT APPLY

Initial: $\neg A \wedge B$

Goal: E

i. H1(P1,P3), H2(P6), H3(P8,P9)

ii. H1(P1,P3), H2(P7), H3(P8,P10)

iii. H1(P1,P2), H2(P4), H3(P8,P9)

iv. H1(P1,P2), H2(P5), H3(P8,P9)

P1: $\neg A \rightarrow A$

P2: $A \rightarrow B$

P3: $A \rightarrow \neg B$

P4: $\neg B \rightarrow A \wedge C$

P5: $\neg B \rightarrow C$

P6: $\neg B \rightarrow A \wedge \neg C$

P7: $\neg B \rightarrow \neg C$

P8: $\neg B \wedge \neg C \rightarrow D$

P9: $D \rightarrow E$

P10: $\neg B \wedge D \rightarrow E$

Trace the state after every primitive action - do not reason from HLA labels alone.

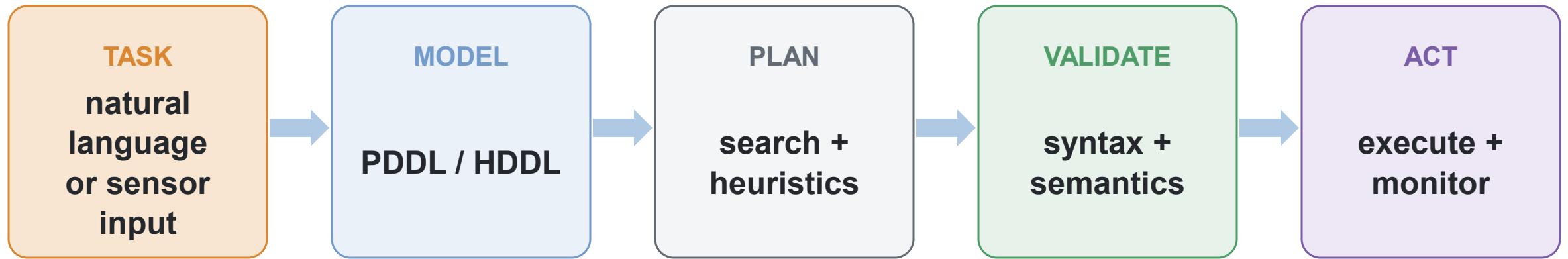


04 Where Planning Goes Next

Modern systems combine symbolic models, hierarchical structure, learned guidance, validation, and execution feedback.

QUESTION → DISCUSSION → REVEAL

■ Modern planning is a loop, not just a solver



feedback repairs the task model, not only the plan

Project mindset: measure each interface - translation, search, validity, and execution.

■ Project: LLM as a translator

Research question

Can feedback turn a plausible model into a correct one?

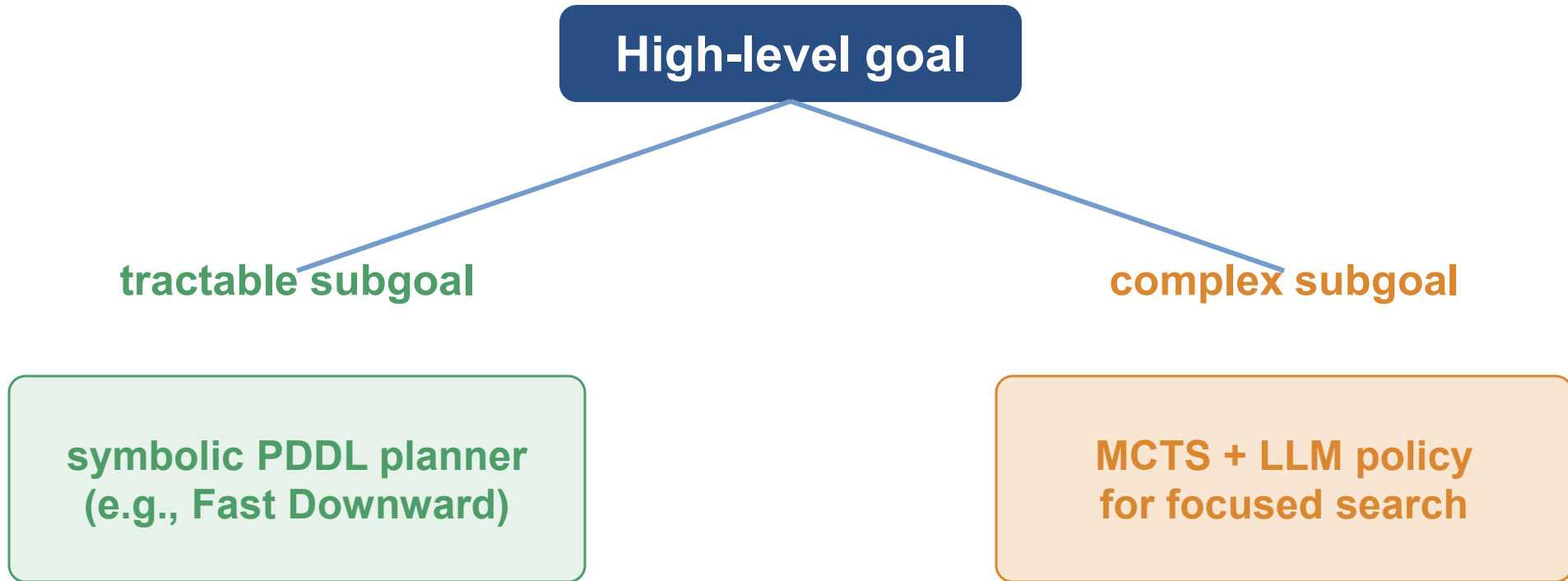
NeurIPS 2024 example
66% average task solve rate
vs 29% GPT-4 CoT baseline



Student-sized experiment

Create 20 natural-language variants of Blocks World. Compare syntax-only checks, plan validation, and environment feedback.

■ Project: LLM aiding complex planning



Evaluate: planning time, success rate, and how the switching rule scales with task length.

■ Project: LLM to come up with heuristics

domain examples → learned heuristic h_θ → guided search

CLASSICAL

[1] LLMs generate domain-specific Python heuristics; evaluate them inside greedy best-first search.

HIERARCHICAL

[2] learns the first heuristic functions for HTN planning; a 2026 preprint explores LLM-generated HTN heuristics.

Student experiment: compare node expansions, coverage, and generalization across problem sizes.

[1] <https://arxiv.org/abs/2503.18809> - Classical Planning with LLM-Generated Heuristics: Challenging the State of the Art with Python Code

[2] <https://arxiv.org/pdf/2605.07707> - Hierarchical Task Network Planning with LLM-Generated Heuristics

■ Project: model one task in PDDL and HDDL

PDDL

state + actions + goal

**search over primitive
action sequences**

same domain

compare

plan length • runtime • nodes •
interpretability

HDDL

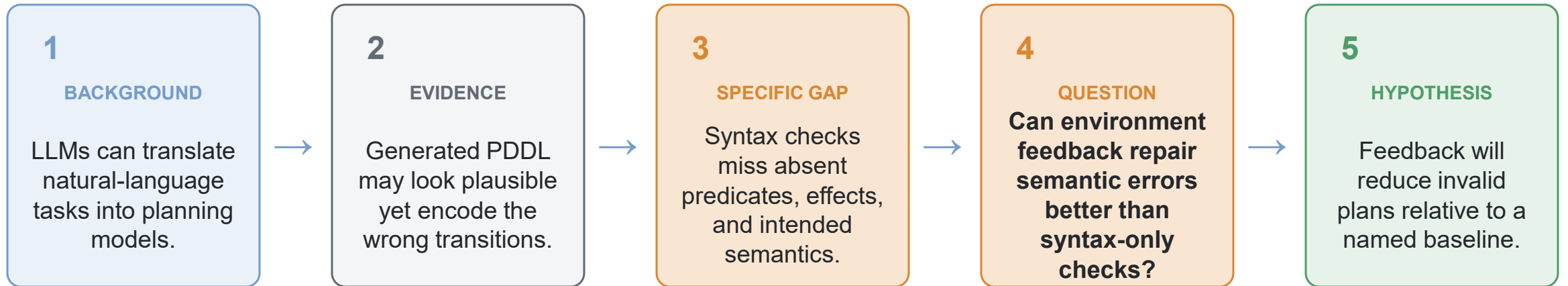
**tasks + methods +
actions**

search over refinements

PANDA supports multiple HTN solving techniques and HDDL as a common hierarchical input language.

■ A strong question shapes the whole project

Scientific questions are built—not discovered in one inspired sentence.



Question template: Can [intervention] improve [measurable outcome] over [baseline] in [setting or failure condition]?

Literature review establishes the background, evidence, gap, and impact. The experiment tests the question and hypothesis.

A strong question makes scope, baselines, evaluation, and the final story much easier to get right.

Example grounded in Mahdavi et al., NeurIPS 2024

■ Hammer first: useful, but not sufficient

Two legitimate ways to begin; one shared standard for deciding whether to continue.

PROBLEM FIRST

**important failure → literature gap
→ candidate method → controlled test**

Start from a need worth solving.

HAMMER FIRST

**method or skill → scan candidate
problems
→ verify a real gap → controlled test**

Start from an asset: LLM, MCTS, heuristic,
representation...

Before committing, make the proposed nail survive three questions

**1 Would the problem matter
without my hammer?**

**2 Why should it beat a
simpler baseline?**

**3 Can success and failure be
measured reproducibly?**

The hammer may start the search; the literature must justify the nail.

Research-design discussion • Apply the same standard to every project idea

■ Five project questions worth testing

- 01 How much feedback is needed to repair LLM-generated PDDL?
- 02 When does an HLA library reduce search - and when does it overconstrain?
- 03 Can a learned heuristic generalize to larger instances?
- 04 Does HDDL improve plan interpretability for human reviewers?
- 05 Can execution failures identify missing predicates or effects?

A good project states one hypothesis, one baseline, one measurable failure mode, and one reproducible evaluation.

■ Where should abstraction live?

in the model

in the heuristic

in the hierarchy



trustworthy action

Take away

PDDL makes assumptions explicit. Relaxations make search practical. HLAs make long plans thinkable.

Read next: AIMA 4e §11.1, §11.3, §11.4, and the Chapter 11 summary (§11.7)